Google Developer Groups

devfest 2022

Bari

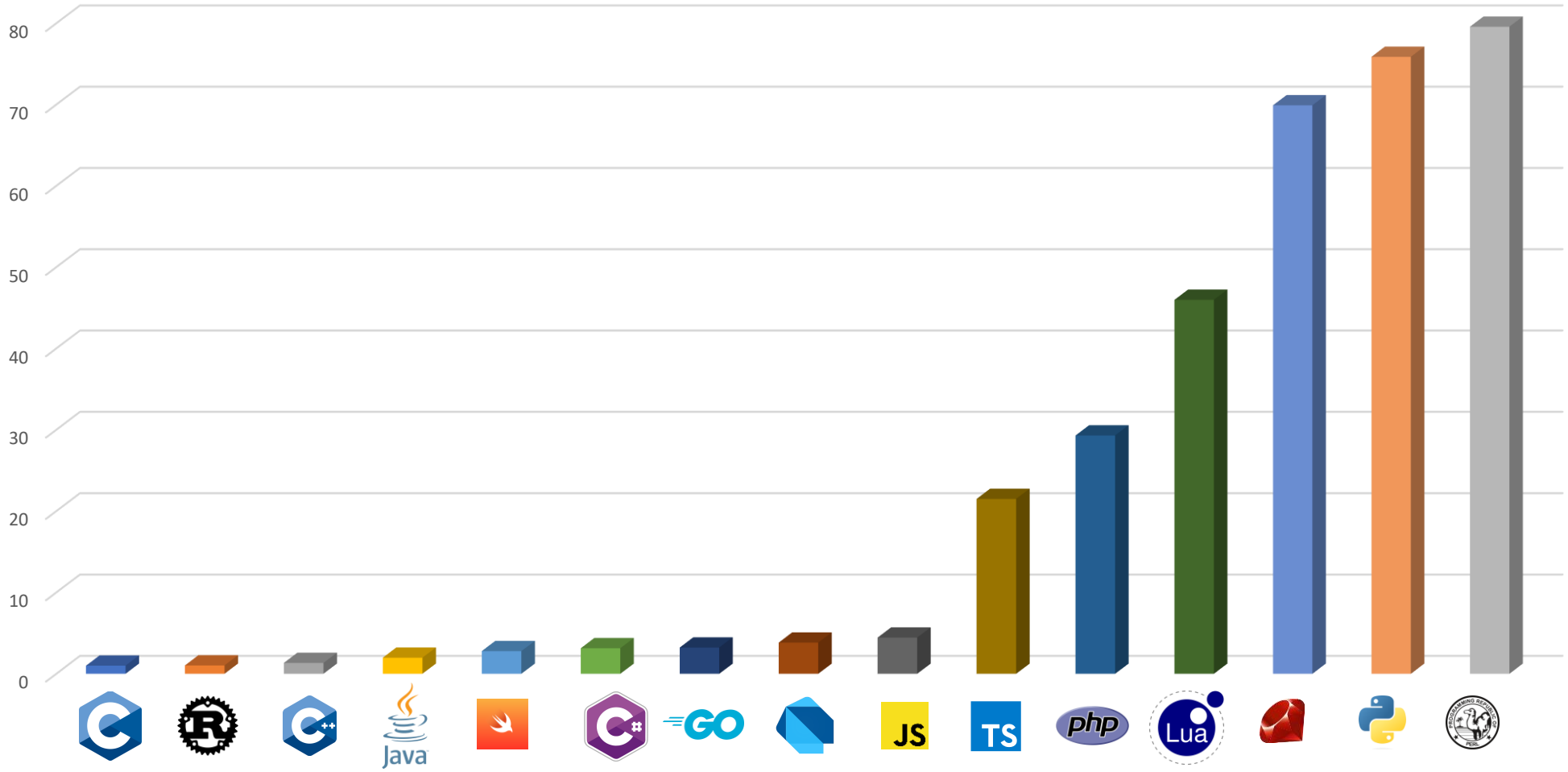
Introduzione a Carbon-Lang

Simone Bonfrate
Software Developer @Wideverse

Google Developer Groups

Bari

Quanto consuma un linguaggio di programmazione?



Linguaggi con uso protetto della memoria

I linguaggi non memory safe sono la causa della maggior parte degli attacchi informatici dovuti alla memoria.

In un comunicato dell'NSA, viene espresso il bisogno di migrare i sistemi da C e C++ a linguaggi che usano in maniera protetta la memoria.

Linux ha iniziato a migrare da C a Rust

Riso Patate e cozze

Performance

Efficienza

Plugin

Framework

Popolarità

OOP

Dobbiamo raggiungere un compromesso

Open Source

Espressività

Community

Multiprocesso

Lavoro

Supporto

Curva di apprendimento

Carbon-lang

Nuovo linguaggio in fase di sviluppo di Google

Performance **paragonabili a C++**

Memory safe

Sintassi **morderna**

Curva di apprendimento **modesta**

Interoperabilità bidirezionale con C++

Obiettivi di Carbon

Scalabilità e sviluppo veloce del software

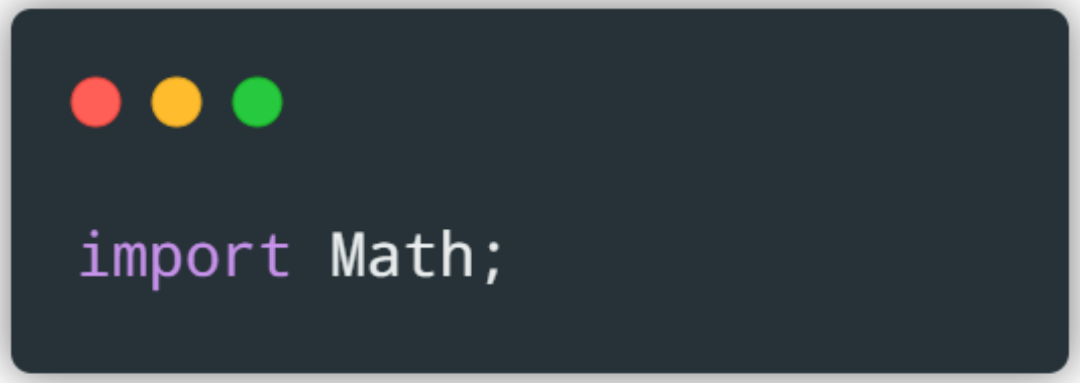
Sviluppo di applicazioni Performance-Critical

Sviluppo di sistemi operativi moderni

Facile lettura, comprensione e scrittura

Diamo uno sguardo alla sintassi

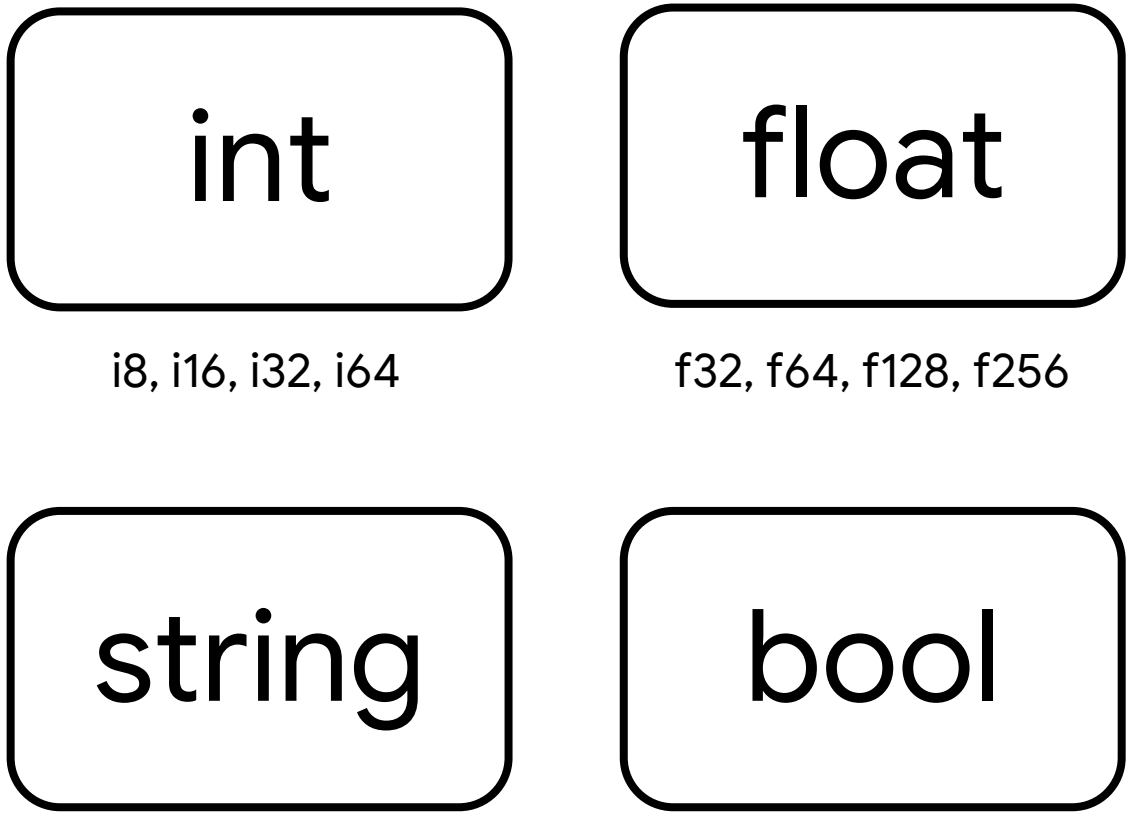
Package imports



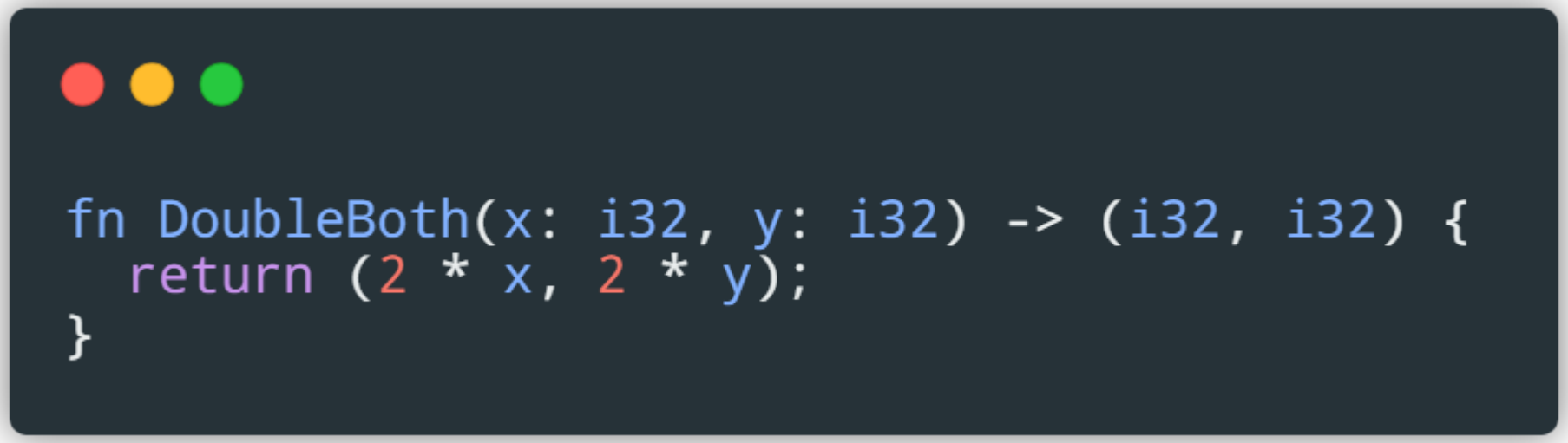
Variabili

```
var x: i32 = 16
x = 32 //ERRORE!!

let y: i32 = 64
y = 32 // OK

let x: auto = "Ciao" //x diventa implicitamente una stringa
```

Tipologie di dato atomico



Tuple



```
fn DoubleBoth(x: i32, y: i32) -> (i32, i32) {
  return (2 * x, 2 * y);
}
```

Struct

```
//Type {.key: String, .value:i32}
var kvpair: auto = {.key = "the", .value = 27};
```

If


if (fruit.IsYellow()) {
Carbon.Print("Banana!");
} else if (fruit.IsOrange()) {
Carbon.Print("Orange!");
} else {
Carbon.Print("Vegetable!");
}

For e while

```
for (var step: Step in steps) {
  if (step.IsManual()) {
    Carbon.Print("Reached manual step!");
    break;
  }
  step.Process();
}
```

```
var x: i32 = 0;
while (x < 3) {
  Carbon.Print(x);
  ++x;
}
Carbon.Print("Done!");
```

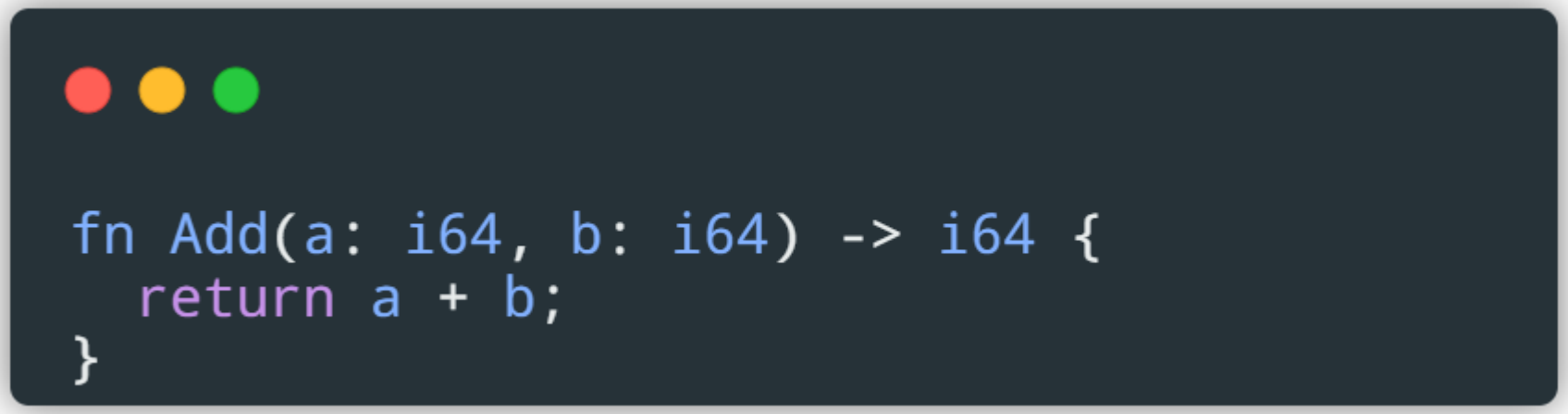
Match

```
fn Bar() -> (i32, (f32, f32));

fn Foo() -> f32 {
  match (Bar()) {
    case (42, (x: f32, y: f32)) => {
      return x - y;
    }
    case (p: i32, (x: f32, _: f32)) if (p < 13) => {
      return p * x;
    }
    case (p: i32, _: auto) if (p > 3) => {
      return p * Pi;
    }
    default => {
      return Pi;
    }
  }
}
```

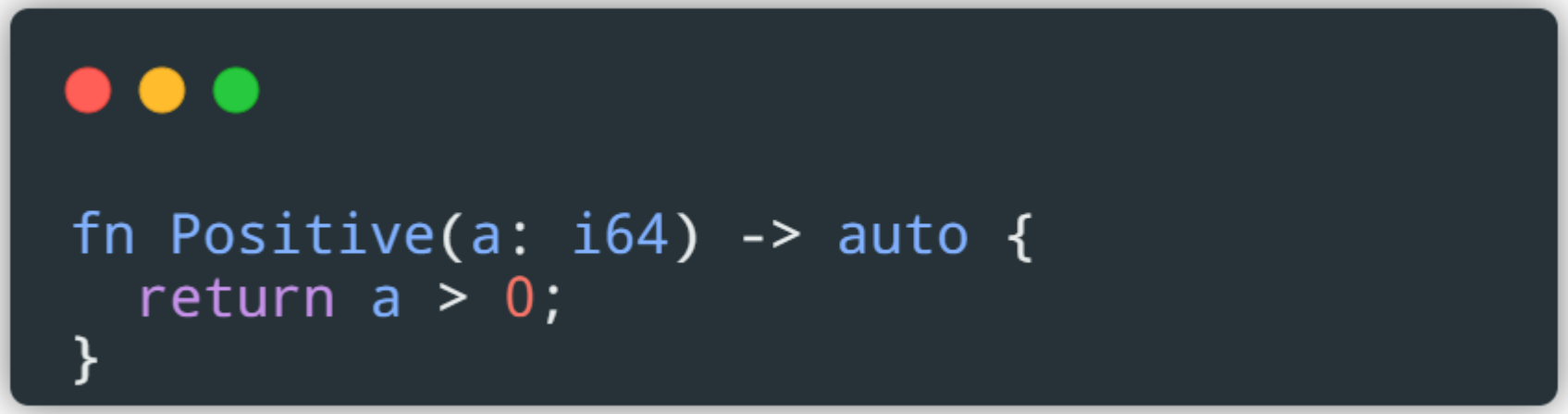
Main function

```
package HelloWorld api;

fn Main() -> i32{
  let msg: String = "Hello World";
  Print(msg)

  return 0;
}
```

Functions



```
fn Add(a: i64, b: i64) -> i64 {
  return a + b;
}
```



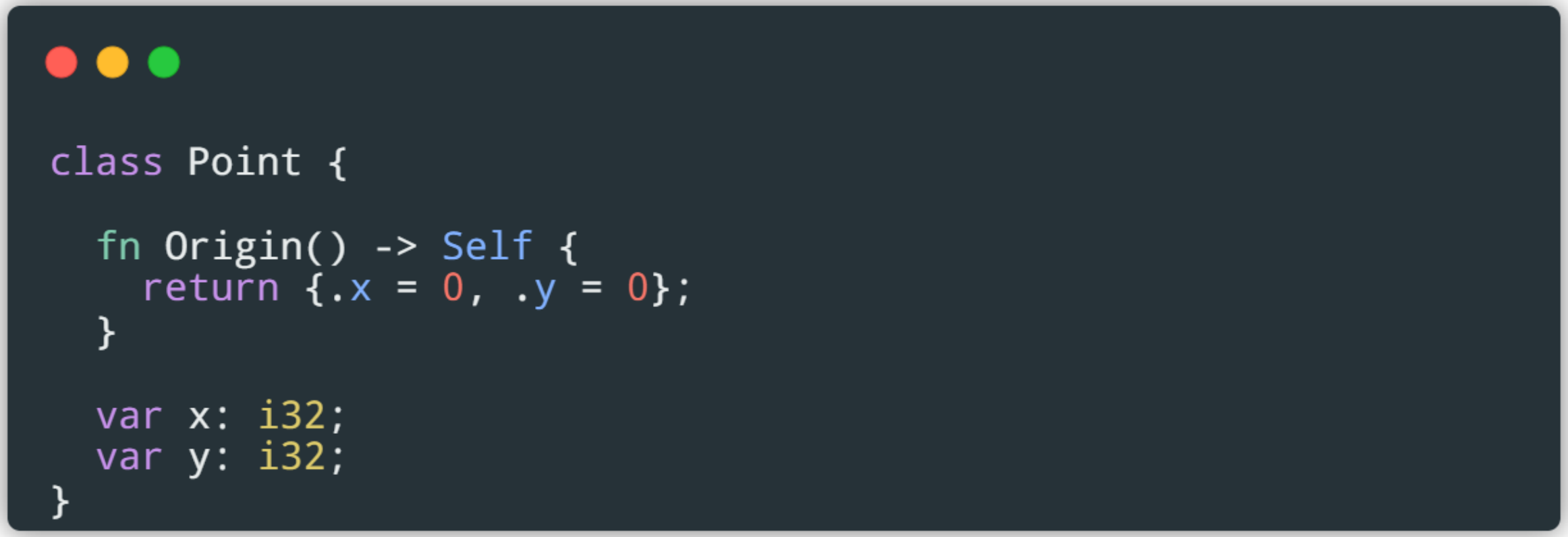
```
fn Positive(a: i64) -> auto {
  return a > 0;
}
```

```
class Widget {
  var x: i32;
  var y: i32;
  var payload: String;
}
```

Classi

```
var sprocket: Widget = {.x = 3, .y = 4, .payload = "Sproing"};
spocket = {.x = 2, .y = 1, .payload = "Bounce"};
```

Factory Function

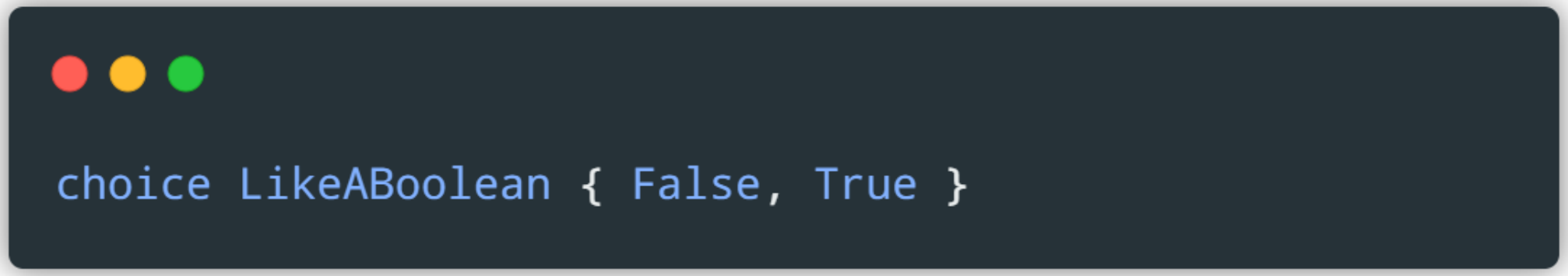


```
class Point {

  fn Origin() -> Self {
    return {.x = 0, .y = 0};
  }

  var x: i32;
  var y: i32;
}
```

Metodi

```
class Point {
  // Method defined inline
  fn Distance[me: Self](x2: i32, y2: i32) -> f32 {
    var dx: i32 = x2 - me.x;
    var dy: i32 = y2 - me.y;
    return Math.Sqrt(dx * dx + dy * dy);
  }
  // Mutating method declaration
  fn Offset[addr me: Self*](dx: i32, dy: i32);

  var x: i32;
  var y: i32;
}

// Out-of-line definition of method declared inline
fn Point.Offset[addr me: Self*](dx: i32, dy: i32) {
  me->x += dx;
  me->y += dy;
}

var origin: Point = {.x = 0, .y = 0};
Assert(Math.Abs(origin.Distance(3, 4) - 5.0) < 0.001);
origin.Offset(3, 4);
Assert(origin.Distance(3, 4) == 0.0);
```

Choice (Enum)



Choice

```
choice IntResult {
  Success(value: i32),
  Failure(error: String),
  Cancelled
}
```

```
match (ParseAsInt(s)) {
  case .Success(value: i32) => {
    return value;
  }
  case .Failure(error: String) => {
    Display(error);
  }
  case .Cancelled => {
   Terminate();
  }
}
```

```
fn ParseAsInt(s: String) -> IntResult {
  var r: i32 = 0;
  for (c: i32 in s) {
    if (not IsDigit(c)) {
      // Equivalent to `IntResult.Failure(...)`
      return .Failure("Invalid character");
    }
    // ...
  }
  return .Success(r);
}
```

Alias

```
class StringPair {
  var key: String;
  var value: String;
  alias first = key;
  alias second = value;
}

var sp1: StringPair = {.key = "K", .value = "1"};
var sp2: StringPair = {.first = "K", .second = "2"};
Assert(sp1.first == sp2.key);
Assert(&sp1.first == &sp1.key);
```

Inheritance

```
base class MyBaseClass {
  virtual fn Overridable[me: Self]() -> i32 {
    return 7;
  }
}

class MyDerivedClass extends MyBaseClass{
  impl fn Overridable[me: Self]() -> i32 {
    return 18;
  }
}
```

Inheritance

```
abstract class MyAbstractClass {
  virtual fn Overridable[me: Self]() -> i32 {
    return 7;
  }
}

class MyDerivedClass extends MyAbstractClass{
  impl fn Overridable[me: Self]() -> i32 {
    return 18;
  }
}
```

E molto altro...

Roadmap

2022:

- Finalizzazione della progettazione del linguaggio
- Rilascio versione 0.1

2023:

- Rilascio versione 0.2

2024-2025:

- Rilascio versione stabile

Domande?

Google Developer Groups

devfest 2022

Bari

Grazie per l'attenzione

Simone Bonfrate

Software Developer @Wideverse

@bonfry

Simone Bonfrate

bonfry.com

Repo
Carbon lang

Google Developer Groups

Bari